RoadSight

Smart Road Construction Progress Monitoring System

Project Report & Testing Readiness Assessment

Project Title:	RoadSight - ML-Based Road Construction Monitoring
Course:	Design Thinking Project
Author:	Ayush Shedame
GitHub:	github.com/ayushshedame-ai/Road-Construction-monitoring-
Branch:	roadsight-prototype
Report Date:	25 October 2026, 11:09 AM IST
Tech Stack:	React 19 + Vite 8 + Leaflet.js + PyTorch + EfficientNet-B0

This report provides a comprehensive technical assessment of the RoadSight prototype, its readiness for testing, identified gaps, and essential changes for production use.

1. Executive Summary

RoadSight is an AI/ML-powered software solution designed for monitoring the physical progress of road construction projects through geotagged site photographs. The system enables Urban Local Bodies (ULBs), state highway agencies (NHAI), and central authorities (MoRTH) to track construction stages without requiring daily technical expert field visits.

The prototype currently includes a fully interactive web-based frontend (React + Vite), a geospatial mapping engine (Leaflet with OSRM road geometries), a client-side computer vision engine for image quality verification, and a Python-based ML training pipeline (PyTorch with EfficientNet-B0). The system covers the complete IRC/MoRTH road construction stage ladder: Earthwork -> GSB -> WMM -> DBM -> BC Asphalt -> Markings.

CURRENT STATUS: The project is at a WORKING PROTOTYPE stage. The frontend UI, interactive mobile simulator, officer dashboard, geospatial map with real road curves, and client-side image analysis all function correctly. However, several critical components require attention before the system can be considered production-ready or suitable for real-world field testing.

2. System Architecture Overview

2.1 Frontend (React + Vite)

The web application is built with React 19 and Vite 8. It features two primary views:
- Field Mobile Simulator: A phone-shaped interface for testing image upload, GPS verification, stage classification, and error scenarios (blur, fraud, geofence violations).
- Officer Dashboard: A full-width analytical dashboard with project cards, progress charts, recent inspection feeds, and a GIS road map panel.

Key UI components: App.jsx, FieldMobileView.jsx (42KB), OfficerDashboardView.jsx (58KB), ProjectMap.jsx (15KB), BeforeAfterSlider.jsx, DatasetExplorerModal.jsx, DesignThinkingModal.jsx.

2.2 GIS / Mapping Engine (Leaflet + OSRM)

Road stretches are plotted on an interactive Leaflet map using high-density polyline coordinates extracted from OSRM (Open Source Routing Machine). Each road stretch has 150-356 GPS waypoints that follow every curve, bend, and interchange of the actual highway/urban road. The rendering uses a Google Maps-style dual-layer ribbon: white outer casing + colored inner core.

Four Pune-area road corridors are included: NH-48 Tathawade Expressway (156 pts), Pune Ring Road Package 3 (356 pts), Baner-Balewadi Link (207 pts), and Katraj-Kondhwa Bypass (208 pts).

Basemaps: OpenStreetMap, Esri World Imagery (satellite), and Esri World Street Map.

2.3 Client-Side CV Engine (cvEngine.js)

A browser-based computer vision engine performs:
- Laplacian Variance Blur Detection: 4-neighbor Laplacian kernel on luminance channel.
- Luminance & Exposure Analysis: Average brightness thresholding.
- pHash Duplicate Detection: 8x8 average luminance perceptual hash + Hamming distance.
- Haversine GPS Geo-fencing: 200m corridor tolerance from project polyline waypoints.
- Color Spectrum Stage Heuristic: Dark pixel (bitumen), gray aggregate (WMM), red-brown (earthwork) percentage-based stage classification.

2.4 Backend ML Engine (Python / PyTorch)

The backend consists of two Python modules:

- ml_engine.py: Production inference pipeline with multi-head EfficientNet-B0 classifier (road_head, duct_head, pedestrian_head), image quality verification, fraud detection, and geo-fence enforcement.
- train_model.py: Full training pipeline with ConvNeXt-Tiny/EfficientNet-B0/ResNet-50 backbones, Ordinal-Penalized Cross-Entropy Loss, CosineAnnealing LR scheduler, and ONNX export.

Current model checkpoint: best_road_stage_efficientnet_b0.pth (15.6 MB, trained on synthetic samples).

3. Feature Readiness Matrix

The following table assesses each feature against its testing readiness:

Feature	Status	Ready?	Blocker
Frontend UI (Mobile + Dashboard)	DONE	YES	None
Image Upload & Preview	DONE	YES	None
GPS Location Display	DONE	YES	None
Leaflet Map with Real Road Curves	DONE	YES	None
Multiple Basemap Layers	DONE	YES	None
Client-Side Blur Detection	DONE	YES	None
Client-Side pHash Fraud Detection	DONE	YES	None
Haversine Geo-fence Validation	DONE	YES	None
Before/After Split Slider	DONE	YES	None
Dataset Registry (13 Free Sets)	DONE	YES	None
Design Thinking Documentation	DONE	YES	None
ML Training Pipeline (Python)	DONE	PARTIAL	Synthetic data only
Multi-Head Stage Classifier Model	BUILT	NO	No real training data
Frontend-Backend API Integration	MISSING	NO	No REST API server
Real ML Inference in Browser/Server	MISSING	NO	Hardcoded mock results
Database / Persistent Storage	MISSING	NO	No database layer
User Authentication	MISSING	NO	No auth system
Progress History Tracking	MOCK	NO	No temporal DB
PDF/CSV Report Export	MISSING	NO	Not implemented
Mobile Camera Integration	MOCK	NO	File upload only

4. Testing Readiness Assessment

4.1 What CAN Be Tested Right Now

READY: Frontend UI & UX Flow
All pages, navigation, mode switching, mobile simulator, dashboard views, and modals work correctly.

READY: Image Quality Checks (Client-Side)
Upload any photo and the blur detector, luminance checker, and pHash engine will process it in real-time.

READY: GPS Geo-fencing Logic
The Haversine distance calculation and 200m corridor enforcement are fully functional with browser Geolocation API.

READY: GIS Map Visualization
All 4 road corridors render with real curved polylines on OSM/Satellite tiles. Layer switching works.

READY: ML Training Pipeline Execution
python backend/train_model.py runs end-to-end with synthetic data, producing a valid .pth checkpoint.

4.2 What CANNOT Be Tested Yet

CRITICAL: Real ML Stage Classification
The frontend cvEngine.js uses pixel color heuristics (dark=asphalt, gray=WMM, brown=earthwork). No actual deep learning model runs in the browser or server. Results are effectively hardcoded.

CRITICAL: Frontend-to-Backend Communication
There is no REST API (FastAPI/Flask) connecting the React frontend to the Python ML engine. The FastAPI dependency is listed in requirements.txt but no server.py or API routes exist.

CRITICAL: Model Accuracy with Real Images
The current checkpoint was trained on synthetic colored noise images (10 per class). It has ZERO accuracy on real construction site photographs.

5. Essential Changes to Make the Project Working

The following changes are ranked by criticality - from MUST-HAVE (project won't work without) to SHOULD-HAVE (significantly improves quality) to NICE-TO-HAVE (polishes the product).

PRIORITY 1: MUST-HAVE (Critical for Working System)

Change #1: Build a FastAPI Backend Server with REST API

WHY: Currently the frontend has NO connection to the Python ML engine. All stage classification results shown in the UI are hardcoded mock values from cvEngine.js. Without a backend API, the ML model cannot actually process uploaded images.

WHAT TO DO:

- Create backend/server.py with FastAPI endpoints:
  POST /api/analyze-image (accept image file + activity_type + GPS coordinates)
  POST /api/compare-progress (accept two images for temporal comparison)
  GET /api/projects (return project list and status)
  GET /api/health (server health check)
- Load the trained PyTorch model on server startup
- Process uploaded images through the actual neural network
- Return real classification results (stage, confidence, completion %)
- Update the React frontend to call these API endpoints instead of cvEngine.js heuristics

EFFORT: ~2-3 days for an experienced developer

Change #2: Collect Real Road Construction Training Images

WHY: The model was trained on synthetic colored noise (10 samples per stage = 60 total). It cannot distinguish real earthwork from asphalt. Without real training data, the entire ML pipeline is a demonstration shell.

WHAT TO DO:

- Minimum viable dataset: 200-500 real images per stage (1200-3000 total across 6 stages)
- Sources for free data:
  * Indian Driving Dataset (IDD) from IIIT Hyderabad - road surface annotations
  * Road Damage Dataset 2022 (RDD2022) - 47K road images from 6 countries
  * Roboflow Universe - search 'road construction', 'asphalt', 'earthwork'
  * Google Images + manual labeling (use the Dataset Registry in the app for links)
  * Personal site visit photography (even 50-100 real images dramatically helps)
- Organize into dataset/train/{stage_folder}/ and dataset/val/{stage_folder}/
- Retrain with: python backend/train_model.py --data_dir ./dataset --epochs 15

EFFORT: ~1-2 weeks for data collection, ~4-6 hours for training

Change #3: Replace cvEngine.js Heuristics with Actual Model Inference

WHY: The current cvEngine.js classifies stages by counting dark/gray/brown pixels. This is a crude color histogram approach that will misclassify most real-world images (e.g., a shadow on GSB gravel will be classified as asphalt because it has dark pixels).

WHAT TO DO (Choose one approach):

Option A - Server-side inference (Recommended):

* Frontend uploads image to FastAPI backend (Change #1)
* Backend runs EfficientNet-B0 inference and returns results
* Frontend displays the real ML prediction

Option B - Browser-side inference with ONNX Runtime Web:

* Export trained model to ONNX (train_model.py already does this)
* Load ONNX model in browser using onnxruntime-web npm package
* Run inference directly in JavaScript without a backend server
* Advantage: Works offline; Disadvantage: Slower, larger download

EFFORT: ~1-2 days

PRIORITY 2: SHOULD-HAVE (Significantly Improves Quality)

Change #4: Add a Database Layer for Project & Inspection History

WHY: All project data is currently in mockData.js - a static JavaScript file. There is no way to persist new inspections, track progress over time, or store uploaded images. Every page refresh resets everything.

WHAT TO DO:
- Use SQLite (simplest, file-based) or Firebase Firestore (cloud-hosted, real-time)
- Tables/Collections needed: projects, inspections, images, users
- Store each inspection with: image_path, GPS, timestamp, ML_result, stage, confidence
- Enable the temporal progress comparison feature with real historical data

EFFORT: ~2-3 days

Change #5: Add User Authentication & Role-Based Access

WHY: The system has no concept of users. In a real deployment, field inspectors should have different permissions than supervising officers or ULB administrators.

WHAT TO DO:
- Implement Firebase Auth or JWT-based authentication
- Roles: Field Inspector (upload images), Supervising Officer (view dashboards), Admin (manage projects)
- Associate each inspection with the authenticated inspector's ID

EFFORT: ~1-2 days

Change #6: Enable Real Device Camera Capture (Not Just File Upload)

WHY: Field inspectors will use the app on mobile phones. Direct camera capture with embedded EXIF GPS metadata is more reliable than separate file upload + manual GPS input.

WHAT TO DO:
- Use <input type='file' accept='image/*' capture='environment'> for mobile camera
- Extract EXIF GPS from the captured JPEG using exifr or piexifjs library
- Auto-populate GPS coordinates from the image EXIF data
- Add a live camera preview using navigator.mediaDevices.getUserMedia()

EFFORT: ~1 day

Change #7: Add Confusion Matrix & Model Evaluation Metrics

WHY: The training pipeline reports only overall accuracy. For a 6-class civil engineering classifier, per-class precision/recall/F1 and a confusion matrix are essential to understand which stages the model confuses (e.g., GSB vs WMM look very similar).

WHAT TO DO:
- After training, run a full validation pass and compute:
  * Per-class Precision, Recall, F1-Score
  * Confusion matrix (6x6)
  * Save metrics to models/evaluation_report.json
- Use sklearn.metrics.classification_report() and confusion_matrix()

EFFORT: ~2-4 hours

PRIORITY 3: NICE-TO-HAVE (Polish & Professional Grade)

Change #8: Progressive Web App (PWA) for Offline Field Use

Add a service worker and manifest.json so inspectors can use the app offline in areas with poor connectivity. Cache the ONNX model locally.

Change #9: PDF/CSV Report Generation from Dashboard

Allow officers to export inspection reports, progress summaries, and chainage completion data as PDF or CSV files.

Change #10: Notification & Alert System

Send email/SMS alerts when anomalies are detected (GPS fraud, duplicate images, stalled progress).

Change #11: Multi-Project & Multi-City Scalability

Currently hardcoded to 4 Pune road corridors. Add functionality to create new projects with custom road stretches, any city, and dynamic OSRM route queries.

Change #12: Admin Panel for Dataset Management

Allow ML engineers to upload new training images, trigger retraining, and view model accuracy from a web dashboard.

6. Project File Inventory

File / Directory	Size	Purpose
src/App.jsx	6.5 KB	Main app shell & navigation
src/components/FieldMobileView.jsx	42 KB	Mobile phone simulator UI
src/components/OfficerDashboardView.jsx	58 KB	Officer analytics dashboard
src/components/ProjectMap.jsx	15 KB	Leaflet GIS map engine
src/components/BeforeAfterSlider.jsx	3.3 KB	Split progress comparison
src/components/DatasetExplorerModal.jsx	7.3 KB	Free dataset registry
src/components/DesignThinkingModal.jsx	8 KB	Design thinking showcase
src/utils/cvEngine.js	10.8 KB	Client-side CV engine
src/data/mockData.js	22 KB	Mock project & inspection data
src/data/realRoadGeometries.json	22 KB	OSRM road polylines (927 pts)
backend/ml_engine.py	10.4 KB	Python ML inference pipeline
backend/train_model.py	13.3 KB	PyTorch training pipeline
backend/requirements.txt	153 B	Python dependencies
models/*.pth	15.6 MB	Trained model checkpoint
dataset/ (train + val)	~1.4 MB	84 synthetic sample images
preview.html	34 KB	Standalone offline preview
MODEL_TRAINING_GUIDE.md	7.3 KB	Training documentation
RoadSight_Project_Report.md	13.5 KB	Full design thinking report

7. Technology Stack Summary

Layer	Technology	Version / Notes
Frontend	React	v19.3.1 (latest)
Build Tool	Vite	v8.3.4
Styling	Tailwind CSS	v4 (CDN loaded)
Icons	Lucide React	v1.61.0
GIS / Maps	Leaflet.js	v1.9.4 + custom ribbons
Route Geometry	OSRM API	Pre-fetched GeoJSON polylines
ML Backbone	EfficientNet-B0	PyTorch pretrained ImageNet
Alt Backbone	ConvNeXt-Tiny	Available in train_model.py
Training	PyTorch	v2.0+ with AMP support
Loss Function	Ordinal-Penalized CE	Custom (distance-weighted)
Image Processing	OpenCV / Canvas API	Blur, luminance, pHash
Backend (planned)	FastAPI	Listed but not implemented

Deployment	ONNX Export	Supported in pipeline

8. Recommended Datasets for Testing & Training

The following freely available datasets can be used to train the model with real road images:

Indian Driving Dataset (IDD)

Source: IIIT Hyderabad | 10K+ frames, Indian road conditions
idd.insaan.iiit.ac.in

Road Damage Detection 2022

Source: IEEE BigData | 47,420 road images, 6 countries
github.com/sekilab/RoadDamageDetector

DeepGlobe Road Extraction

Source: CVPR Challenge | 6,226 satellite tiles, 50cm res
competitions.codalab.org

Mapillary Vistas v2.0

Source: Meta / Mapillary | 25,000 street-level with road segmentation
mapillary.com/dataset/vistas

CrackForest-dataset

Source: GitHub | 118 annotated road crack images
github.com/cuilimeng/CrackForest-dataset

GAPs / Asphalt Pavement

Source: Eisenbach et al. | 1,969 pavement distress patches
gaps-netzwerk.de

KITTI Vision Benchmark

Source: Karlsruhe IT / Toyota | 80K+ frames, 3D calibrated road data
cvlibs.net/datasets/kitti/

Cityscapes

Source: Daimler AG | 5,000 fine + 20,000 coarse urban road annotations
cityscapes-dataset.com

9. Quick-Start Testing Guide

Step 1: Start the Frontend

cd "C:\Users\Ayush\OneDrive\Attachments\Desktop\ROAD site"
npm run dev

Open http://localhost:5173/ in your browser.

Step 2: Test the Mobile Simulator

Click the 'START ASSESSMENT' button in the top bar. Select an activity type (Road Construction, Utility Ducts, or Pedestrian Infrastructure). Upload a test image or use the built-in demo presets in the dropdown selector.

Step 3: Test Error Scenarios

Select preset #4 (Blurry Image), #5 (GPS Violation), or #6 (Duplicate Fraud) from the dropdown to see how the system handles quality failures and anomaly detection.

Step 4: Test the Officer Dashboard

Switch to 'Officer Dashboard' mode using the pill switcher in the header. Explore the project cards, progress charts, GIS map, and recent inspections feed.

Step 5: Run the ML Training Pipeline

```
cd backend
pip install -r requirements.txt
python train_model.py --model efficientnet_b0 --epochs 5
```

This runs end-to-end training on the sample dataset and saves a checkpoint to models/.

10. Conclusion & Recommendation

The RoadSight prototype successfully demonstrates the viability of using machine learning and computer vision for automated road construction progress monitoring. The interactive UI, geospatial mapping with real road geometries, and the ML training pipeline are all functional and showcase the complete vision of the solution.

However, the project is currently a DEMONSTRATION PROTOTYPE, not a production-ready system. The three most critical gaps are:

1. No actual ML inference - the frontend shows hardcoded/heuristic results, not real model predictions
2. No backend API server - the Python ML engine and React frontend are completely disconnected
3. No real training data - the model was trained on synthetic noise images with zero accuracy on real photos

To transition from prototype to a testable working system, the minimum required effort is:
- Build a FastAPI backend server (~2-3 days)
- Collect 1200+ real road construction images (~1-2 weeks)
- Retrain the model on real data (~4-6 hours GPU time)
- Connect the frontend to the backend API (~1-2 days)

With these changes, the system would be ready for controlled field testing with real construction site inspectors in a pilot deployment.

Report generated on 25 October 2026, 11:09 AM IST
RoadSight - Smart Infrastructure | Design Thinking Project